

Dependency and Scope: what the formalisation now proves

11 September 2026

This note summarises a round of work on the Isabelle/HOL formalisation of *Dependency and Scope*. It is meant to be read before the sources. The short version: the translation theorem is now proved outright rather than conditionally, all three impossibility results have been carried from the compact calculus to the exact HL₋ types that mirror the *How Logic Works* checker, and no defect was found in the algorithms.

1 What changed

The previous state of the development proved a great deal about the compact calculus and about *accepted* outputs of the exact translator, but it did not prove that the exact translator succeeds. Its own status note said so: “a general proof that the exact emitter succeeds on every valid source is still a separate obligation.” That obligation is now discharged, and with it the remaining exact-layer counterparts of L2–L4.

Four new theories carry the work: `LF_HLW_Nesting` (structural well-formedness of emitted proofs), `LF_HLW_Fresh` (what the eigenconstant repair does and does not change), `LF_HLW_Erasure` (rule transfer through the erasure δ_H), `LF_HLW_Construct` (the construction theorem), and `LF_HLW_Theorem10` (the impossibility results).

2 The construction theorem

Theorem (`hlPaperCorrect_toFitch`, in `LF_HLW_Construct`). *For every nonempty Lemmon proof P in the original paper’s correctness class there are a route and an F with*

$$\begin{aligned} \text{hlLemmonToFitchChecked } P &= \text{Inr}(\text{route}, F), & \text{hlFitchCorrect } F, \\ \text{hlFitchConclusion } F &= \text{hlConclusion } P, \\ \text{set}(\text{hlFitchPremises } F) &\subseteq \text{set}(\text{hlOpenPremises } P). \end{aligned}$$

In words: the checked translation always succeeds, and what it returns is a Fitch proof that passes every check, proves the same conclusion, and assumes nothing the source did not already assume. This is stronger than the earlier guarantee, which said only that *if* the translator returns something, that something is correct. An output guard of that kind cannot rule out the translator rejecting a perfectly good source; this theorem does.

The route is left existential on purpose. The checked operation tries the direct algorithm first and falls back to the derivation-tree construction, and since Conjecture 27 is false no single direct algorithm can be claimed. The theorem is about the translation as a whole, which is the honest form.

How it is proved. The chain is: unfold the source to a derivation tree (this was already proved), emit a Fitch proof from the tree, and then show that erasing the emitted proof gives back a correct Lemmon proof. The last step is where the work was. Two inductions over the emitter, each covering all twenty-one rules, do it:

- `hlEmitDerivationFuel.ruleOK` — every emitted line passes the dependency-based rule check after erasure;
- `hlEmitDerivationFuel.scopeOK` — every emitted line passes the *scope*-based check as well, where the eigenconstant conditions are stated against the assumptions genuinely in Fitch scope.

The second does not follow from the first. A line’s Fitch scope contains every root premise and every enclosing box head, so it is in general *larger* than that line’s dependency set, and the obvious antitonicity argument runs the wrong way. What makes it provable is discussed in §4.

3 The impossibility results at the exact types

Theorem 10 and Conjectures 27 and 28 were proved only for the compact `proof/fitch` datatypes. They now hold for the exact ones too, in the same form:

<code>hl.theorem_10</code>	both witnesses, each correct and each with no image
<code>hl.conjecture_27_false</code>	no injective renumbering of the witness helps
<code>hl.conjecture_28_false</code>	a correct source with no duplication-free image

We expected this to be the hard part, and planned either to port the compact positional apparatus (102 lemmas about scope *paths*, and the definitions they need) or to build a bridge between the two layers. Neither was needed. The exact nesting checker already encodes the same content in a different way: it admits a discharge citation only when the pair is a box that has closed *at the citing line’s own level*, and it tracks a visible set that a closed box’s interior never re-enters. Three structural facts about boxes follow directly, and each of the three obstructions is one of them.

Boxes never cross (`hlBoxSpans_nested`) If one box runs from line a to line c and another starts at a' with $a < a' \leq c$, then the second ends at $c' \leq c$. Example 11 is the obstruction:

deps	line	formula	justification
1	(1)	P	A
2	(2)	Q	A
1,2	(3)	$P \wedge Q$	$\wedge I$ 1,2
2	(4)	$P \rightarrow (P \wedge Q)$	CP 1,3
	(5)	$Q \rightarrow (P \rightarrow (P \wedge Q))$	CP 2,4

An image would need a box from 1 to 3 and another from 2 to 4. The second starts inside the first and ends outside it. Since $1 < 2 \leq 3$, the fact forces $4 \leq 3$. Four lines of Isabelle.

Nothing outside a box cites into it (`hlFitchNestingFrom_no_cite_into_box`) A line numbered past a box’s last line cannot cite a number inside it. Example 12 is the obstruction:

deps	line	formula	justification
1	(1)	P	A
2	(2)	Q	A
1	(3)	$P \vee R$	$\vee I$ 1
2	(4)	$Q \wedge Q$	$\wedge I$ 2,2
	(5)	$Q \rightarrow (Q \wedge Q)$	CP 2,4
1	(6)	$(P \vee R) \wedge (Q \rightarrow (Q \wedge Q))$	$\wedge I$ 3,5

Line 5 discharges assumption 2 at line 4, so any image has a box from 2 to 4. Line 3 is numbered between them, so it is written inside that box — its dependency set is irrelevant to this. Line 6 cites line 3 and is numbered past the box. Nothing is stranded by the dependency bookkeeping; it is the page layout that fails.

Two boxes never share a last line (`hlBoxSpans_same_last`) This is the obstruction behind both conjectures:

deps	line	formula	justification
1	(1)	P	A
2	(2)	Q	A
1,2	(3)	$P \wedge Q$	$\wedge I$ 1,2
2	(4)	$P \rightarrow (P \wedge Q)$	CP 1,3
1	(5)	$Q \rightarrow (P \wedge Q)$	CP 2,3

Both conditional proofs discharge *from line 3*, one releasing assumption 1 and the other assumption 2. Nothing in the Lemmon notation objects. A Fitch proof would need two boxes both ending at line 3. Note that this is a third phenomenon, distinct from the other two: here the boxes (1, 3) and (2, 3) nest perfectly, and no line is trapped. Conjecture 27 then falls because sharing a discharge is a property of the proof rather than of the page — `hlSharedDischarge_permute` shows it survives every injective renumbering, so no permutation of this proof has a positional image either.

4 Two things we learned about the checker

The witness inference is what makes the scope check provable. After the emitter’s repair, a universal introduction’s eigenconstant must avoid every constant in the surrounding Fitch scope, not merely those in the subderivation’s own open assumptions. The repair renames exactly the constants of the conclusion that are missing from the goal and occur in the scope, so the question is whether an inferred eigenconstant is always one of those. It is, and the reason is in *your* checker rather than in ours: `hlWitnessLists` draws its candidate constants from `hlConstantsInFormula q`, the constants of the *premise*. So an inferred witness always occurs in the premise, and having been abstracted away it never occurs in the conclusion — which is exactly the set the repair empties. Had the candidates been drawn from anywhere else, the

scope check would not have gone through, and we would have been looking at a genuine gap rather than a proof.

hlConcludesAtTop is load-bearing. The requirement that a subproof’s body end with a line rather than with another subproof looks like a presentational nicety. It is not. Without it a box whose body ends with a box inherits that box’s last line, and “two distinct boxes never share a last line” is simply false — so Conjectures 27 and 28 would not be refutable by this route. The condition the nesting checker imposes on every body is exactly what rules that case out. We have recorded this in the worklist so it is not mistaken later for a structural triviality.

5 One hypothesis, and where it goes

The construction theorem needs one side condition on the derivation tree: no assumption label may carry two different formulas (`hlAssumptionsFunctional`). The layout environment is consulted by `hlEnvironmentLine`, which returns the *first* entry with a given label, so without this the environment need not describe the tree’s open assumptions at all.

It is automatic for anything coming from a proof — a label is a line number and a line carries one formula — and `hlToDerivation_functional` discharges it: the unfolding puts the open assumptions inside `hlSourceAssumptions`, which is functional because `hlLookupLine` is. So it does not appear in the statement of the theorem above. We mention it because it is a real condition on arbitrary `hl.derivation` values, and anyone building trees by other means should know it is there.

6 What is checked, and how to check it

The complete 49-theory development processes from Isabelle’s plain HOL image in about six minutes:

```
isabelle build -d . -e Lemmon_Fitch
```

or, for editing, `isabelle jedit -d . -n -l HOL Lemmon_Fitch.thy`. Isabelle 2025-2. No project heap is needed and none is built.

A source scan finds no `sorry`, no `oops`, no axiomatizations, no oracles and no `quick_and_dirty` settings anywhere in the sources, and every maintained theory is explicitly imported by the `cap`, so processing the `cap` really does check all of it. The Haskell and SML exports regenerate byte-for-byte unchanged, so the recorded executable results still apply: 9,874 differential comparisons with 0 disagreements, 860 of 860 rejection messages exact, all 80 cases of the supplied regression suite passing against a generated-kernel shim, and 26 of 26 public API assertions.

7 What is not done

The web application has not been adapted to the generated Aeson and parser interface; that is interface engineering rather than formalisation, and `HASKELL_AUDIT.md` should be read before treating the generated module as a drop-in replacement. `PROJECT_GOALS.md` lists the few questions we decided deliberately not to pursue, with the reason in each case.

No defect was found in the emitter during this round. The sixteen findings in the supplied Haskell recorded earlier in `HASKELL_AUDIT.md` stand as they were.